# ATOM User Guide

---

**Version 6.2.0, October 2026**

---

José Luís Martins
INESC MN, Lisboa, Portugal

jlmartins@inesc-mn.pt

jose.l.martins@tecnico.ulisboa.pt

Section 4 adapted from SIESTA documentation by Alberto García

# Contents


**1 Introduction** **2**

1.1 What the code does . . . 2

1.2 Resources for ab-initio DFT and pseudopotentials . . . 2

**2 Installation** **3**

2.1 Step 1: Downloading and extracting the archive . . . 3

2.2 Step 2: Generating the documentation . . . 3

2.3 Step 3: Compiling the code . . . 4

**3 Running the code** **5**

3.1 Validation . . . 5

3.2 Running pseudopotential construction with all tests and plots . . . 5

3.2.1 First command line argument . . . 6

3.2.2 Second and third command line arguments . . . 6

3.3 Running step by step . . . 7

3.4 The all-electron calculation . . . 7

3.5 Constructing the pseudopotential . . . 10

3.6 Visualizing the pseudopotential . . . 12

3.7 Applying the Kleinman-Bylander transformation. . . . 13

3.8 Visualizing the KB operators. . . . 13

3.9 Logarithmic derivatives plots. . . . 14

3.10 Testing the pseudopotentials . . . 14

**4 The atom.dat input file** **15**

4.1 All-electron calculations . . . 15

4.2 Pseudopotential generation . . . 15

4.3 Core corrections . . . 17

4.4 Format of the file . . . 17

**5 Computational details** **20**

5.1 Radial grid . . . 21

5.2 Solving radial equations . . . 21

5.3 Atomic orbitals . . . 22

5.4 Fourier transforms . . . 23

# 1 Introduction

This is a modernized version of an old code that generates and tests pseudopotentials to be used in Density Functional Theory (DFT) calculations of electronic structure of molecules, solids, etc...

The original code was written by Sverre Froyen at the University of California Berkeley. Phys. Rev. B 26, 1738 (1982), https://doi.org/10.1103/PhysRevB.26.1738. A 1985 version of the code can be found in `Doc/Historic/atomsve.f`, it can still be compiled and run! That code had major enhancements by Norman J. Troullier at the University of Minnesota, Minneapolis. Phys. Rev. B 43, 1993 (1991), https://doi.org/10.1103/PhysRevB.43.1993 Since then the code has been maintained and improved by José Luís Martins. Peter Schuster and Manuel Maria Alemany wrote new plotting packages. GGA were introduced by Carlos Balbas and José Soler in a collaboration with the SIESTA code.

The present version is a "translation" from old FORTRAN (Fortran 70) to modern Fortran (90 $\rightarrow$ present), with detailed documentation and added enhancements by José Luís Martins. Another modernized version of the original Sverre Froyen code can be found at `https://siesta-project.org/siesta/Documentation/Pseudopotentials`, is maintained by Alberto García, who added some features and made substantial structural changes to the April 1990 (5.0) Minnesota version. Both versions can use the same input file as the original 1982 code.

## 1.1 What the code does

The program's basic capabilities are:

- All-electron DFT atomic calculations for arbitrary electronic configurations and a few Local Density Approximation (LDA) and Generalized Gradient Approximations (GGA) functionals.
- Generation of ab-initio pseudopotentials, and their transformation to the Kleinman-Bylander form with atomic basis functions.
- Atomic calculations in which the effect of the core is represented by a previously generated pseudopotential. These are useful to make sure that the pseudopotential correctly reproduces the all-electron results for the valence complex.

## 1.2 Resources for ab-initio DFT and pseudopotentials

We will not review here the background of density functional pseudopotentials. But, in this case more than ever, there is a lot to be gained from reading the original literature, for both the basics, and what is implemented in the code. Here are a few basic references:

- The book by Richard Martin "Electronic Structure: Basic Theory and Practical Methods" (Cambridge University Press) reviews the field and has a chapter on pseudopotentials.
- Some seminal papers on ab-initio pseudopotentials, (but certainly not an exhaustive or historic list of the development of the key concepts.):

  Hamann, Schluter, Chiang, Phys. Rev. Lett. 43, 1494 (1979)
  Kerker, J. Phys. C 13, L189-94 (1980)
  Bachelet, Hamann, Schluter, Phys. Rev. B 26, 4199 (1982)

- Troullier-Martins elaboration of Kerker method:

  Troullier, Martins, Phys. Rev. B 43, 1993 (1991)

- Partial core corrections:

  Louie, Froyen, Cohen, Phys. Rev. B 26, 1738 (1982)

- Reviews of plane-wave pseudopotential ab-initio calculations:

  W. E. Pickett, "Pseudopotential Methods in Condensed Matter Applications", Computer Physics Reports 9, 115 (1989)

  M. C. Payne, M. P. Teter, D. C. Allan, T. A. Arias and J. D. Joannopoulos, "Iterative minimization techniques for ab initio total-energy calculations: molecular dynamics and conjugate gradients", Rev. Mod. Phys. 64, 1045, (1992)

# 2 Installation

## 2.1 Step 1: Downloading and extracting the archive

The code is available from José Luís Martins GitHub:

`https://github.com/jlm785/pseudopotential`

If you downloaded the `pseudopotential-6.x.y.tgz` file (where `x` and `y` are major and minor version numbers) just extract it,

```
$ tar xzf pseudopotential-6.x.y.tgz
```

and you will have the code in a `pseudopotential-6.x.y` directory If you cloned the git you already have the relevant code in the main directory. As always releases are stable, git clones are just snapshots. Do not clone if you notice active modifications.

## 2.2 Step 2: Generating the documentation

The documentation is in the `atom/Doc` directory, where `atom/` stands for `pseudopotential-6.x.y`. It includes this file, `user_guide.tex` in `atom/Doc/UserGuide`, with a description of the key input file `atom.dat` and key output and intermediate files. To get the pdf just run

```
atom/Doc/UserGuide$ pdflatex user_guide.tex
```

and repeat to have the correct references,

```
atom/Doc/UserGuide$ pdflatex user_guide.tex
```

to get a `user_guide.pdf` file.

A navigable description of the whole code for developers can be generated from the information in the folder `Doc/doxy`. For that detailed description you need doxygen (`https://www.doxygen.nl`) and graphviz (`https://graphviz.org/`). Both are available on most distributions. To check that your computer has doxygen and graphviz installed run the commands

```
atom/Doc$ dpkg -s doxygen
```

```
atom/Doc$ dpkg -s graphviz
```

on a Debian based distribution (Ubuntu et al), in other distributions use equivalent tools (yum, rpm, dnf, zypper,...) or the relevant GUI.

If they are not available just install them

```
atom/Doc$ sudo apt install doxygen
```

```
atom/Doc$ sudo apt install graphviz
```

again for Debian based distros, use equivalent tools (yum, rpm, dnf, zypper,...) or the relevant GUI. Finally run doxyfile in the doxy directory

```
atom/Doc/doxy$ doxygen Doxyfile
```

You will get two folders `atom/Doc/html` and `atom/Doc/latex` If you know what you are doing you can edit `atom/Doc/doxy/Doxyfile` to change the defaults. Finally it is useful to create a link to the file `atom/Doc/html/index.html`

```
atom/Doc$ ln -s html/index.html documentation.html
```

Opening that link in your browser will allow you to see the documentation for every file in the present code, with the relevant call tree and a short description of all the arguments of the subroutines.

### 2.3 Step 3: Compiling the code

The code is regularly tested with several Fortran compilers, `ifx` (from Intel oneAPI), `gfortran` (from gnu). Once in a while it also tested with other compilers, such as `pgfortran`. As the performance is not an issue for these "simple" one-dimensional (radial Schrödinger) calculations just use the compiler that is more familiar for you.

The good news is that the code is self-contained, it just needs the compiler,

```
atom$ cd Src
```

```
atom/Src$ gfortran atom_all.f90 lib_*/*.f90 lib_blas_pack/*.f -o atom_all.exe
```

will work out of the box. But you may want to change the compilation flags, for example to optimize for your computer. That is easily achieved with `make`. All the folders have an appropriate `Makefile` you just need edit the preamble file `atom/Src/make.inc`. In principle you have to make just two choices. The first is to identify which CPU you are using, as it narrows the choice of possible compilers. This is done by commenting/uncommenting the lines below `# compiler for the job`. The second is the compiler you want, and is the key decision. Again comment/uncomment the lines below `# Suggestions for compilers`. For any other modifications of the file it is assumed you know what you are doing... The CPW2000, `elk` and WANNIER90 use a `make.inc` file so you can find plenty of examples in those codes for specific combinations of computers and compilers.

To compile just go to `atom/Src` and type make

```
atom/Src$ make
```

In a few seconds you should have the main library `libatom_`*compiler*`.a` and a few executables `atom_`*compiler*`.exe`, etc... where *compiler* is the name of the chosen compiler. As you probably are not interested in comparing compilers you can create links, for example

```
atom$ ln -s Src/atom_all_gfortran.exe atom_all.exe
```

```
atom$ ln -s Src/atom_gfortran.exe atom.exe
```

etc...

The code relies on `gnuplot` (`http://www.gnuplot.info`) to produce graphics interactively. Just install it on your machine (apt, yum, rpm, dnf, zypper,...) if you want to see them as the code runs. If `gnuplot` is not installed, there is no problem, you will have in your folder the data required to make plots with `gnuplot` or with `matplotlib` on another machine.

# 3 Running the code

I assume you have a working directory separate from the source. In the following I will call it `atom/WORK`, and assume you created the links in `atom/`.

This is a code with a long history. Keeping it back-compatible (the original code still runs) made it easier to check that modifications did not introduce bugs, at the cost of some compromises, mainly in the number and constraints on the format of files. The old code and its additions communicated through files. The new version of the code still uses those file formats of several decades ago. The good news about the present version, is that, except for their presence in the working directory, you do not have to worry about them in most cases.

You can run the whole pseudopotential generation and testing procedure at once, or run step by step. Each step is one of the rectangular boxes in Fig. 1, the whole generation and testing procedure runs all the steps in a single shot. As the code uses intermediary files you can go back and repeat each of the steps you want, for example to modify the default values of parameters or generate new graphics.

**WARNING:** the defaults of the code are "educated guesses" for the majority of atoms. Any pseudopotential should be thoroughly tested before being used.

## 3.1 Validation

The compiled code should be tested. Go to the `Validation` folder and run the validation script (edit the path to the executable in `run.sh` if you did not compile with `ifx`),

```
atom$ cd Validation
```

```
atom/Validation$ bash run.sh
```

go for a coffee, and when you return you should have the pseudopotentials for all atoms with the default choices. **WARNING:** the defaults of the code are "educated guesses" for the majority of atoms, in particular transition metals, Lanthanides, and Actinides. Any pseudopotential should be thoroughly tested before being used!

## 3.2 Running pseudopotential construction with all tests and plots

Assuming you have gnuplot installed and you copied the executables to the code root, you can run the whole pseudopotential generation and testing with a simple command:

```
atom/WORK$ ../atom_all.exe
```

the code will check if there is a `atom.dat` file in the working directory. This is the original input file of the old code and its strictly formatted structure is described in Section 4. If the file is present it will use it, otherwise the program will ask for the chemical symbol of the element you want, write a `atom.dat` file with default values. In both cases it will then proceed with the reference all-electron calculation, the pseudopotential generation, finding atomic orbitals basis set and non-local projectors, generate plots and perform tests on the pseudopotential.

After running the code you will find many files on your working directory. These are described in more detail in the next subsections. Briefly, you get the "input" file `atom.dat` and the detailed output file `atom.out`. The most important files are those that are read by other codes, listed in Table 1, where *Xy* is the chemical symbol.

Table 1: Pseudopotential files written for other codes.

| file | code |
|---|---|
| *Xy*`_LDA_TM.UPF`, *Xy*`_GGA_TM.UPF` | QUANTUM ESPRESSO |
| *Xy*`.psp8` | ABINIT |
| *Xy*`.psml` | SIESTA and other codes that use libpsml |
| *Xy*`.psf` | SIESTA |
| *Xy*`_POTKB_F.DAT` | CPW2000 |
| *Xy*`_POTRE.DAT` | PARSEC |

Notice that *Xy*`.psf` and *Xy*`_POTRE.DAT` are very similar. The first part of the internal file `psd.pot` is similar to those, however it includes at the end the information about the partial core kinetic energy density. Finally you get many graphics related files, `*.pdf` and `*.gp`, and a few `*.dat` internal data files. For details see the description of running the code step by step below.

### 3.2.1 First command line argument

You can pass arguments in the command line, for example

```
atom/WORK$ ../atom_all.exe -h
```

will print a short help text and stop.

You can also run in a full interactive mode

```
atom/WORK$ ../atom_all.exe -x
```

where you are asked to override the default values, and therefore can be cumbersome. Overriding the defaults is easier by running individually each main component of the code. The interactive mode is described below in the section for the step by step run. In the interactive mode a "wrong answer" will cause the program to ignore it and proceed with the default value, eventually giving a second chance to the user to enter an answer in the accepted range.

The most useful first argument is the chemical symbol,

```
atom/WORK$ ../atom_all.exe Si
```

where you can swap "Si" for the chemical symbol of any element that interests you from Hydrogen to Oganesson. All the work will be done with default values without prompting the user. Read the **Warning** above about default values.

### 3.2.2 Second and third command line arguments

Default values for the reference atomic configuration and pseudopotential radii can be overridden by careful editing the `atom.dat` file. Although the choice of using the partial core-correction for exchange and correlation can be altered in that way, it is easier to do it by the second argument of the command line, that takes the values `yes/no` force using or not using the partial core correction.

The code generates "SIESTA-like" confined atomic orbitals. By default two basis sets are

built and written to the *Xy*_POTKB_F.DAT file (and to plot_kb.dat): set 1 is double-zeta with polarization (DZP) and set 2 is single-zeta (SZ). The second zeta is the relaxed energy derivative $\phi$ of the first one. The atomic orbitals have angular momenta $l = 0, \ldots, l_a$, where $l_a$ is the larger of the highest occupied $l$ and $l_{\max} - 1$, $l_{\max}$ being the highest angular momentum of the pseudopotential, and the polarization orbitals have $l_a + 1$ (not above $f$). There is a single polarization orbital (in SZP and DZP), the perturbative polarization of the first $l_a$ orbital $\psi$ as in SIESTA. The third argument, which takes the values SZ/SZP/DZ/DZP (lower case is accepted), replaces the two default sets by a single set of that kind; any other value stops the program. As the arguments are positional, the second one must be given to use the third, for example ../atom_all.exe Si no DZ. See section 5.3 for the details on how these orbitals are constructed.

### 3.3 Running step by step

While atom_all runs everything at once, the same results can be obtained by a few smaller programs that communicate through files as described in the next subsections. Running them step by step makes it easy to repeat one of the steps, for example with other core radii or another local component, without repeating the others. Figure 1 shows the programs and the files each of them reads and writes.

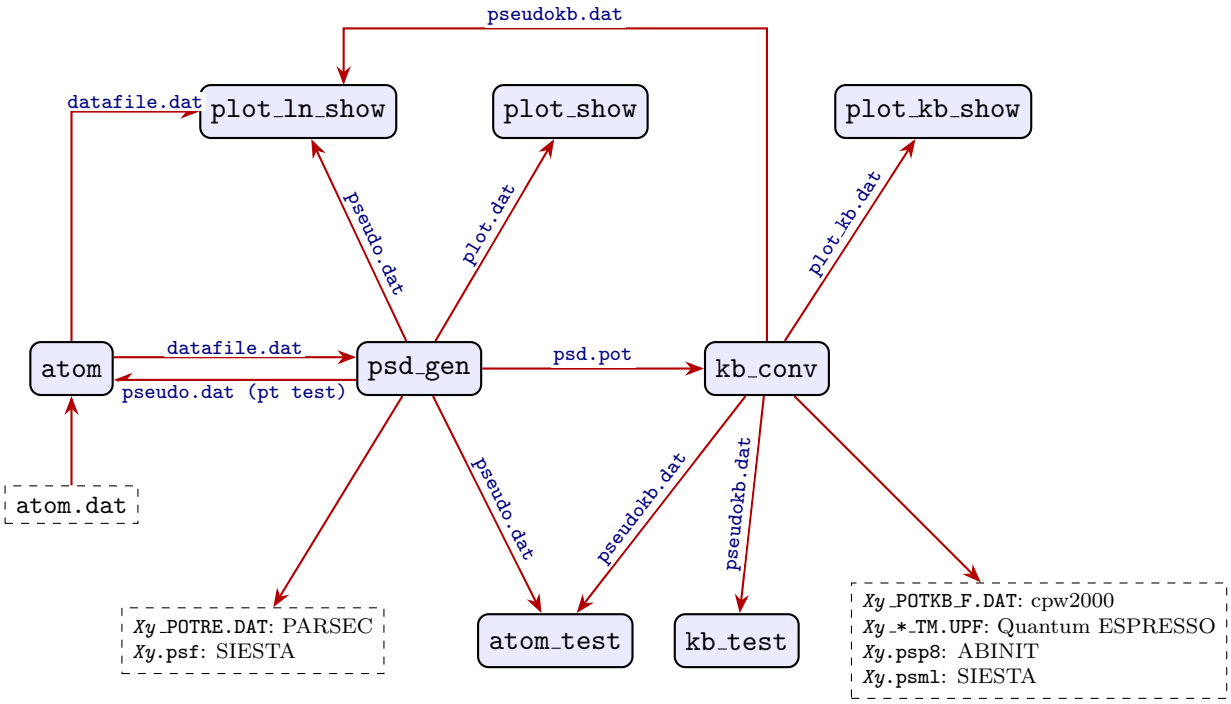



Figure 1: Programs (boxes) and the files through which they communicate when the code is run step by step. atom, psd_gen, atom_test and kb_test read atom.dat; the dashed boxes at the bottom list the files written for other codes (Table 1). atom_all runs the whole chain at once.

### 3.4 The all-electron calculation

The starting point of a pseudopotential construction is an all-electron calculation. This is done by running the "atom" code:

```
atom/WORK$ ../atom.exe
```

the program searches for a file `atom.dat` in the `WORK` directory with the information on the atomic configuration and options of the calculation such as the exchange and correlation model. This is the key input file and is described in Section 4.

If the file `atom.dat` is not present, the code asks the user the chemical symbol of the element to be calculated, writes a `atom.dat` file with the default values and runs the calculation. The user may edit the `atom.dat` file to change the default choices, taking into account that the file is for fixed format read (back-compatibility).

If the validation step of Section 3.1 was run, the user can find `atom.dat` for all atoms in the folders `Validation/res_`$Xy$ where $Xy$ is the chemical symbol. Modifying this file is the starting point for the construction of a good pseudopotential.

At the end of the `atom.exe` calculation a file `datafile.dat` is written with the information of the atomic configuration and self-consistent potential to be used by the pseudopotential generation code. In the interactive mode the output goes to the default output, in contrast to the non-interactive all at once mode where the output goes to the `atom.out` file. The output is largely self-explanatory. All energies are in Rydberg unless explicitly stated, and all lengths in Bohr radii. At the top of the example of the output shown below, we have: the information on the date the code was run; the choice of the correlation functional, "ca" meaning the LDA based on Ceperley-Alder homogeneous electron gas results; The core and electronic charge, and the atomic configuration with quantum numbers and occupation (`occ`)

```
 atom-LDAca04-Nov-26           Si      file generated by atm_write 04-Nov-26
**********************

 Si pseudopotential generation
 -----------------------------

 correlation = ca   nonspin-polarized

 nuclear charge             = 14.000000
 number of core orbitals    =  3
 number of valence orbitals =  3
 electronic charge          = 14.000000
 ionic charge               =  0.000000


 input data for orbitals

  i    n    l    s     j     occ

 1    1    0   0.0   0.0    2.0000
 2    2    0   0.0   0.0    2.0000
 3    2    1   0.0   0.0    6.0000
 4    3    0   0.0   0.0    2.0000
 5    3    1   0.0   0.0    2.0000
 6    3    2   0.0   0.0    0.0000
```

Next we have the description of the radial mesh used in the calculation,

```
 radial grid parameters
```

```
r(1) = .0 , r(2) =  0.887E-06 , ... , r(2686) = 119.814
a =  6.000  b = 200.000
```

indicating that we have 2686 points on the grid, the first being the origin, the second $r(2)$ at $0.887 \times 10^{-6}$ bohr, and the last at almost 120 bohr. See subsection 5.1 for how the radial grid is constructed using the values of $a$ and $b$.

For each orbital we have a list

- `a extr`, the value of the wave-function $\psi$ where $|r\psi|$ has an extremum;
- `r extr`, the radius of that extremum;
- `r zero`, the radii where $\psi = 0$ (nodes);
- `r 90/99 %`, the radii that contain 90% and 99% of the charge of the orbital.

For the $3p$ orbital of Si one obtains

```
n = 3  l = 1  s = 0.0
       a extr          0.264  -0.620
       r extr          0.316   2.146
       r zero          0.718
       r 90/99 %       4.463   6.726
```

showing extrema at 0.316 and 2.146 bohr, and a node at 0.718 bohr. It also shows that 90% of the electron density in that orbital is within a 4.463 radius and 99% within 6.726 bohr.

Next we have the eigenvalues, kinetic energy and Coulomb potential energy of the orbitals (in Rydberg),

```
Total number of iterations needed for electron screening potential is 16



 Si output data for orbitals
 ---------------------------

 nl    s      occ         eigenvalue    kinetic energy      pot energy

 1s   0.0    2.0000    -130.36911374     183.01377674    -378.73491523
 2s   0.0    2.0000     -10.14892755      25.89954327     -71.62102263
 2p   0.0    6.0000      -7.02876330      24.42537956     -68.74331327
 3s   0.0    2.0000      -0.79662747       3.23745219     -17.68692632
 3p   0.0    2.0000      -0.30705182       2.06135800     -13.62572590
 3d   0.0    0.0000       0.00231324       0.00232980      -0.38199473
```

and finally a control of the virial theorem. For non-relativistic calculations the virial sum should be zero. For relativistic calculations this (non-relativistic) theorem does not hold. In this case we see that the virial sum is close to zero ($-0.00018$ Ry, about 2.5 meV).

```
virial theorem(nonrelativistic)
 --------------

 kinetic energy  *  2      =     1149.95307571
 potential energy          =    -1151.36048794
 virial correction         =        1.40722738
 ---------------------------------------------
 virial sum                =       -0.00018485
```

### 3.5 Constructing the pseudopotential

The key operation is the construction of the pseudopotential. To generate the Troullier-Martins pseudopotential run

```
atom/WORK$ ../psd_gen.exe
```

The program reads the all-electron results from the `datafile.dat`, presents a short summary of the results of the all-electron calculation and the current values for core radii, and asks the user if they should be modified.

```
   Found a file with all-electron results for Si


   Calculation was done with Ceperley-Alder LDA


   Based on a non-spin-polarized and non-relativistic calculation


   Calculation was run on 04-Nov-26 with code version 6.1.0



   The current value of core radii are:

   l =  0  rc =        1.800
   l =  1  rc =        1.800
   l =  2  rc =        1.800

   Do you want to modify them? (y/n)
```

if you answer yes you will be presented with suggested core radii from the `atom.dat` file and from an internal table of default values, plus data on maxima and last node of the valence wavefunctions. It then asks for the maximum angular momentum of the pseudopotential channels, and indicates the accepted range. If you give a value out of the accepted range, the program will proceed with the default value.

The next question is about the partial core correction for the exchange and correlation, the user is presented with the current situation, and is asked if they want to change it. Next the user is asked if they want to change the radius where the partial core sets in. The reference radius is the outermost point $r_{\rm ref}$ where the core and valence electron densities are equal, $\rho_{\rm core}(r_{\rm ref}) =$

$\rho_{\rm valence}(r_{\rm ref})$. The radius for the partial core correction is $r_{\rm pcc} = c_{\rm fac} r_{\rm ref}$, the default value is $c_{\rm fac} = 1$ (`cfac` in the question). For more details see section 4.3.

The code will next ask if the user wants to change the radius for the partial core kinetic energy density, $r_{{\rm pc}\tau} = c_{{\rm fac}\tau} r_{\rm ref}$, and in case of an affirmative answer, the user would enter $c_{{\rm fac}\tau}$ (cfactau).

Once the questions are answered, the code will print the parameters that describe the Troullier-Martins pseudopotential, and the truncated (partial) kinetic energy,

```
Si pseudopotential generation using the Improved Troullier and Martins method
 ------------------------------------------------------------

 nl    s    eigenvalue      rc          cdrc       delta       gamma       alpha_j

 1s   0.0   -0.796627    1.796507    0.369768   -1.387513    0.708147   -0.100295   -0.135680
 2p   0.0   -0.307052    1.796507    0.206765   -0.943628   -0.236975   -0.008022    0.021175
 3d   0.0    0.002313    1.796507    0.000000   -8.732549   -0.883722   -0.086774    0.195276
 core kinetic energy =    564.38

 kinetic energy density truncated
 inside r =  1.204
 ac =  6.954 bc = -4.133 cc =  0.491

 truncated core kinetic energy =     63.31
```

it will then list the characteristics of the pseudo-wavefunctions, as described in the previous section,

```
n = 1  l = 0  s = 0.0
        a extr         0.737
        r extr         1.778
        r zero
        r 90/99 %      3.306   4.835

 n = 2  l = 1  s = 0.0
        a extr         0.620
        r extr         2.146
        r zero
        r 90/99 %      4.463   6.726

 n = 3  l = 2  s = 0.0
        a extr         0.150
        r extr        80.446
        r zero
        r 90/99 %     99.081 111.157
```

and the core radii (bohr), eigenvalues (Rydberg), and pseudopotential values at origin and minimum (more negative) values.

```
 test of eigenvalues

 rcut =  s   1.80  p   1.80  d   1.80
```

```
 eval =  -0.79661  -0.30704   0.00231



 l    vps(0)    vpsmin      at r

 s     0.065    -5.501     1.392
 p    -6.064    -6.064     0.000
 d   -15.757   -15.757     0.000
```

Next it will show the eigenvalue, kinetic energy and bare potential energy of the pseudo-wavefunctions,

```
Si output data for orbitals
 ----------------------------

 nl    s      occ         eigenvalue    kinetic energy      pot energy

 1s   0.0    2.0000      -0.79661328       0.51034020      -3.73919585
 2p   0.0    2.0000      -0.30703675       0.76934634      -3.25652725
 3d   0.0    0.0000       0.00231324       0.00232980      -0.10914135
```

the eigenvalues should be the same as the all-electron calculation. Finally it will show the energy contributions for the pseudo-atom.

```
 total energies
 --------------

 sum of eigenvalues        =       -2.20730006
 kinetic energy from ek    =        2.55937308
 el-ion interaction energy =      -13.99144619
 el-el  interaction energy =        5.86809482
 vxc    correction         =       -2.51140668
 virial correction         =        0.17637063
 exchange + corr energy    =       -1.92764767
 kinetic energy from ev    =        2.55936317
 potential energy          =      -10.05099904
 -------------------------------------------
 total energy              =       -7.49163587
```

At the end of the calculation the user has five new files (back compatibility), an unformatted `pseudo.dat` file used later in tests, a `plot.dat` file used for later plotting, a formatted `psd.pot` file used to pass the information for the Kleinman-Bylander transformation, and the files *Xy*`_POTRE.DAT` for PARSEC and *Xy*`.psf` for SIESTA. The file `psd.pot` is therefore the key output file of the program.

### 3.6 Visualizing the pseudopotential

If `gnuplot` is installed the user can compare pseudo and all-electron wave-functions, observe the pseudopotential and respective Fourier transforms by running

```
atom/WORK$ ../plot_show.exe
```

The program will read `plot.dat` (or another file chosen by the user) and allow the user to change the range of the axis of the plots, and save a `pdf` version of the plots. If the user doesn't clean the `*.gp` files at the last question, the `WORK` directory will have several `command_`*name*`.gp` and *name*`.gp` files. The user can see the plots again with gnuplot

```
atom/WORK$ gnuplot command_name.gp
```

and can edit the command file to change the plot. The user can also use the data in *name*`.gp` files to generate plots with other software, including xmgrace and matplotlib. The script `Tools/atom_plot_mpl.py` draws the same figures with matplotlib from the `fig_`*name*`.dat` files. If gnuplot is not installed, the program just quits.

### 3.7 Applying the Kleinman-Bylander transformation.

In plane-wave codes the pseudopotential is used in its Kleinman-Bylander form (Kleinman and Bylander, Phys. Rev. Lett. 48, 1425 (1982)). That transformation plus the generation of an efficient atomic basis set can be achieved by

```
atom/WORK$ ../kb_conv.exe
```

the program will read the `psd.pot` file and ask the user if they accept the defaults and in the negative case allows the choice of new parameters.

The key decision here is the choice of the local component of the pseudopotential. A bad choice can introduce "ghost states" and the code will check if they appear, with the method of Gonze, Käckell and Scheffler, Phys. Rev. B 41, 12264 (1990). If a ghost state is found you are strongly warned in the output. As a rule of thumb, choose the least attractive potential as the local part; if you still have trouble, try larger core radii.

At the end of the calculation the program writes an unformatted `pseudokb.dat` used later in tests, a `plot_kb.dat` file used for later plotting (which also contains the atomic basis functions), and the files for other codes: *Xy*`_POTKB_F.DAT` for CPW2000, *Xy*`_LDA_TM.UPF` (or `_GGA_`) for QUANTUM ESPRESSO, *Xy*`.psp8` for ABINIT and *Xy*`.psml` for SIESTA, where *Xy* is the chemical symbol of the relevant element.

The program also suggests the range of energy cutoffs for the plane-wave calculations. It is a range because different properties converge at different rates.

### 3.8 Visualizing the KB operators.

If `gnuplot` is installed the user can compare the pseudo wave-functions with the atomic basis functions, observe the KB operators and respective Fourier transforms, and have an estimate of the rate of convergence of the total energy with plane-wave basis size by running

```
atom/WORK$ ../plot_kb_show.exe
```

The program will read the `plot_kb.dat` file or another file specified by the user. See the section on visualizing the pseudopotential for dealing with the files generated by the program. The basis functions can also be plotted, one figure for each basis set and angular momentum, with `python3 Tools/plot_basis.py` (matplotlib), which reads `plot_kb.dat`.

### 3.9 Logarithmic derivatives plots.

If `gnuplot` is installed the user can generate the plots of the logarithmic derivatives of the wave-functions with respect to energy for a given radius with

```
atom/WORK$ ../plot_ln_show.exe
```

The program will read the `pseudo.dat`, `pseudokb.dat` and `datafile.dat` files (or the files specified by the user). See the section on visualizing the pseudopotential for dealing with the files generated by the program.

This is a test based on scattering theory. Two potentials scatter in the same way at a radius $r_0$, for an energy $E$ and angular momentum $l$, if the logarithmic derivatives

$$\frac{\partial}{\partial r} \log \psi_l(r; E)\Big|_{r_0} \tag{1}$$

of the solutions $\psi_l(r; E)$ of the radial Schrödinger equation that are regular at the origin are the same. Norm conservation (the never cited Topp-Hopfield theorem) makes sure that, for $r_0$ outside the core radii, the logarithmic derivatives of the all-electron and pseudo potentials agree at the reference eigenvalues and are very close near them. The plots show how far in energy that agreement extends, for the semi-local and the Kleinman-Bylander forms. A ghost state shows up as an extra pole of the KB curve.

### 3.10 Testing the pseudopotentials

Pseudopotential testing means using the pseudopotential in an environment that is different from the one used to generate it, and for which we have reliable data without the pseudopotential approximation. The simplest test is the atom in other configurations. For example after generating a pseudopotential for silicon with the $s^2p^2$ ground state configuration, one compares all-electron and pseudopotential calculations for the ionized configurations $s^1p^2$ and $s^2p^1$, and for the excited configurations $s^1p^3$, $s^1p^2d^1$ and $s^2p^1d^1$.

By editing the `atom.dat` file and running

```
atom/WORK$ ../atom.exe
```

the user can either see the all electron results for a different configuration (`ae` keyword on the first line and deleting the second line, back-compatibility as you should have guessed) or see the result of a different configuration on the pseudopotential described in `pseudo.dat`. Running

```
atom/WORK$ ../kb_test.exe
```

the user achieves the same type of test for the KB pseudopotential described by `pseudokb.dat`. If you are worried about ghost states, this is the ultimate test for their absence. Note that `atom.exe` always rewrites `datafile.dat`, also in a pseudopotential test (`pt`); keep a copy of the all-electron one if you want to run `psd_gen.exe` or `plot_ln_show.exe` again.

A series of tests of ionization and promotion energies can be done with the original `atom.dat` by running

```
atom/WORK$ ../atom_test.exe
```

At the end the program will print a comparison of the ionization energies and promotion energies with all- electrons pseudopotential and pseudopotential in KB form.

Discrepancies on the range of a few hundredth of eV are to be expected, anything larger may indicate a bad pseudopotential.

# 4 The atom.dat input file

The description of that file, Section 4, was mostly written by Alberto García for the SIESTA documentation (version 3.2.2, January 2006) and adapted to describe the present code.

## 4.1 All-electron calculations

Assume we want to find the orbital eigenvalues, total energy, and/or charge density of Si in its ground state for some functional. You already have the compiled executables from SIESTA or from the ATOM code that you can find in the `Src/` folder. You should now go to a "working" directory and try the following: Our input file is named `atom.dat` and contains the lines (see Sect. 4.4 for more details):

```
  ae     Si   ground state all-electron
n=Si c=ca
     14.0       0.0       0.0     120.0       6.0      80.0
   3    2
   3    0     2.00      0.00
   3    1     2.00      0.00

#234567890123456789012345678901234567890123456789012345678901234567890      Ruler
```

We can run the calculation by typing *pathtoexecutables*`/atom.exe > atom.out` or *pathtoexecutables*`/atom_`*compilername*`.exe > atom.out` Here we are redirecting the output to `atom.out`. As described in Section 2.3, the present release of the code adds the compiler name by default, for example `atom_gfortran.exe` for code compiled with `gfortran`.

It is interesting to peruse the `atom.out` file. In particular, it lists the orbital eigenvalues (in Rydbergs, as almost every other energy in the program):

```
 nl    s      occ         eigenvalue    kinetic energy      pot energy

 1s   0.0    2.0000    -130.36911227     183.01377610    -378.73491457
 2s   0.0    2.0000     -10.14892691      25.89954256     -71.62102165
 2p   0.0    6.0000      -7.02876265      24.42537869     -68.74331196
 3s   0.0    2.0000      -0.79662738       3.23745216     -17.68692614
 3p   0.0    2.0000      -0.30705175       2.06135786     -13.62572529
```

(For a relativistic or spin-polarized calculation, there would be "up" and "down" flags in the `s` column above.)

## 4.2 Pseudopotential generation

To generate a pseudopotential for Si, using the Troullier-Martins scheme repeat the calculation with relativistic effects (although they are quite small) and use the LDA (Ceperley-Alder flavor). The input file contains the lines (see Sect. 4.4 for more details):

```
  pg     Si     Silicon
       tm2
```

```
n=Si c=car
     14.0       0.0       0.0     120.0       6.0      80.0
   3    4
   3    0     2.00      0.00
   3    1     2.00      0.00
   3    2     0.00      0.00
   4    3     0.00      0.00
   1.90     1.90     1.90     1.90

#2345678901234567890123456789012345678901234567890123456789012345678901234567890      Ruler
```

Note the two extra lines with respect to an all-electron calculation. The pseudopotential core radii for all channels are 1.90 bohrs. Even though they are nominally empty in the ground state, we include the $3d$ and $4f$ states in order to generate the corresponding pseudopotentials.

It is certainly cumbersome to type a fixed format input file. The good news is that running `/atom_`*compilername*`.exe` in a folder without the `atom.dat` and entering the chemical symbol of the element will generate such a file with default values, which can be modified for subsequent runs. Notice that the default values may not be the best, so that file is just a template to be edited!

We can run the calculation by typing again *pathtoexecutables*`/atom.exe > atom.out` or *pathtoexecutables*`/atom_`*compilername*`.exe > atom.out` The `atom.out` file now has the spin-orbit effect and we see in the effect on eigenvalues that is small. The empty scattering orbitals may have a small positive eigenvalue.

```
nl    s      occ         eigenvalue    kinetic energy      pot energy

1s   0.5    2.0000    -130.51541377       0.00000000    -380.32924747
2s   0.5    2.0000     -10.18537055       0.00000000     -71.99156489
2p  -0.5    2.0000      -7.05518951       0.00000000     -69.08320077
2p   0.5    4.0000      -7.00775181       0.00000000     -68.71538886
3s   0.5    2.0000      -0.79937163       0.00000000     -17.74263422
3p  -0.5    0.6667      -0.30807126       0.00000000     -13.66178946
3p   0.5    1.3333      -0.30567131       0.00000000     -13.60785806
3d  -0.5    0.0000       0.00231324       0.00000000      -0.38199583
3d   0.5    0.0000       0.00231324       0.00000000      -0.38199582
4f  -0.5    0.0000       0.00340155       0.00000000      -0.35430233
4f   0.5    0.0000       0.00340155       0.00000000      -0.35430233
```

Running the pseudopotential generation by typing *pathtoexecutables*`/psd_gen.exe > psd.out` or *pathtoexecutables*`/psd_gen_`*compilername*`.exe > psd.out` will give you (just accept the defaults) a new default output where you can find the pseudopotential eigenvalues.

```
nl    s      occ         eigenvalue    kinetic energy      pot energy

1s   0.5    2.0000      -0.79936317       0.50555149      -3.74109919
2p  -0.5    0.6667      -0.30805322       0.77243757      -3.26354801
2p   0.5    1.3333      -0.30566087       0.76702417      -3.25195654
3d  -0.5    0.0000       0.00231324       0.00233363      -0.10914167
3d   0.5    0.0000       0.00231324       0.00233363      -0.10914167
```

```
 4f  -0.5    0.0000       0.00340158       0.00342993      -0.10122710
 4f   0.5    0.0000       0.00340158       0.00342993      -0.10122710
```

which are almost identical to the all-electron values.

### 4.3 Core corrections

The program can generate pseudopotentials with the non-linear exchange-correlation correction proposed in S. G. Louie, S. Froyen, and M. L. Cohen, Phys. Rev. B 26, 1738 (1982).

In the traditional approach (which is the default for LDA calculations), the pseudocore charge density equals the charge density outside a given radius $r_{pc}$ had the smooth form

$$\rho_{pc}(r) = Ar\sin(br)$$

inside that radius. A smooth matching was provided with suitable $A$ and $b$ parameters calculated by the program.

A new scheme has been implemented to fix some problems in the generation of GGA pseudopotentials. The smooth function is now

$$\rho_{pc}(r) = r^2 \exp\left(a + br^2 + cr^4\right)$$

and derivatives up to the second are continuous at $r_{pc}$.

To use core corrections in the pseudopotential generation the jobcode in the first line should be `pe` instead of `pg`.

The radius $r_{pc}$ should be given in the sixth slot in the last input line (see above). If it is negative or zero (or blank), the radius is then computed using the fifth number in that line (`rcore_flag`, see the example input file above) and the following criterion: at $r_{pc}$ the core charge density equals `rcore_flag`*(valence charge density). It is *highly recommended* to set an explicit value for the pseudocore radius $r_{pc}$, rather than letting the program provide a default.

If `rcore_flag` is input as negative, the full core charge is used. If `rcore_flag` is input as zero, it is set equal to one, which will be thus the default if `pe` is given but no numbers are given for these two variables.

The seventh number in that line plays the role of `rcore_flag` for the partial core kinetic energy density. If it is zero or blank it is set equal to one. If it is negative the full core kinetic energy density is used, which is only useful for tests with very large cutoffs.

The output file contains the radius used and the $A$ ($a$) and $b$ (and $c$) parameters used for the matching.

### 4.4 Format of the file

For historical reasons, the input file is in a rigid column format. Fortunately, most of the column fields line up, so the possibility of errors is reduced. We will begin by describing in detail a very simple input file for an **all-electron calculation** for the ground state of Si.

The file for Si could be:

```
   ae     Si   ground state all-electron
n=Si c=ca
```

```
   0.0       0.0       0.0       0.0       0.0       0.0
3    2
3    0     2.00      0.00
3    1     2.00      0.00

#2345678901234567890123456789012345678901234567890      Ruler
```

- The first line specifies:
  - The calculation code (`ae` here stands for "all-electron").
  - A title for the job (here `Si ground state all-electron`).

  (format 3x,a2,a50)

- Second line:
  - Chemical symbol of the nucleus (here `Si`, obviously)
  - Exchange-correlation type. Here, `ca` stands for Ceperley-Alder. The "best" LDA choice should be `ca`. It is also possible to use a gradient-corrected functional: `pb` indicates use of the PBE scheme by Perdew, Burke, and Ernzerhof (PRL 77, 3865 (1996)). The exchange-correlation energy and potential are computed using a modified Soler-Balbás package.
  - The character `r` next to `ca` is a flag to perform the calculation relativistically, that is, solving the Dirac equation instead of the Schrodinger equation. The full range of options is:
    - `s` : Spin-polarized calculation, non-relativistic.
    - `r`: Relativistic calculation, obviously polarized.
    - (blank) : Non-polarized (spin ignored), non-relativistic calculation.

  (format 3x,a2,3x,a2,a1,2x)

- Third line. Its use is somewhat esoteric and for most calculations it may contain just zeros (six numbers) as the code will use the default values.

  The first number is the nuclear charge, $Z$. You can invent elements with fractional charge. If it is zero the program will get it from the chemical symbol of the previous line. The second and third number are there for historical reasons. Their values are never used. The fourth to sixth numbers specify the integration mesh. Again if they are zero the code uses default values. The fourth number is the maximum range of the radius values $r_{\max}$. The sixth number $B$ controls the increment between consecutive grid points. $r(n+1) \sim (1+1/B)r(n)$, in other words it takes about $B$ points to increase the radius in the mesh by about $e$. The fifth number $A$ controls the first non-zero point in the mesh. $r(1) = 0$, $r(2) \simeq exp(-A)/(Z*B)$.

  The full specification of the quasi-logarithmic mesh is $a = exp(-A)/Z$, $b = 1/B$, $r(i) = a*(exp(b*(i-1))-1)$, $r(i) < r_{\max}$.

The rest of the file is devoted to the specification of the electronic configuration:

- Fourth line:
  Number of core and valence orbitals. For example, for Si, we have $1s$, $2s$, and $2p$ in the core (a total of 3 orbitals), and $3s$ and $3p$ in the valence complex (2 orbitals).

  (format 2i5)

- Fifth, sixth... lines: (there is one line for each valence orbital)
  - `n` (principal quantum number)
  - `l` (angular momentum quantum number)
  - Occupation of the orbital in electrons.

  (format 2i5,3f10.3)

  (There are two f input descriptors to allow the input of "up" and "down" occupations in spin-polarized calculations (see example below)). There is a third float point number that can fix the eigenvalue of the orbital. It has not been used in years, Forget it exists.

  The valence orbitals can be given in any order. The same orbital cannot be given twice, nor can a valence orbital be one of the core orbitals (the program stops in those cases). Two valence orbitals with the same angular momentum and different principal quantum numbers are allowed.

The different treatment of core and valence orbitals in the input for an all-electron calculation is purely cosmetic. The program "knows" how to fill the internal orbitals in the right order, so it is only necessary to give their number. That is handy for heavy atoms... Overzealous users might want to check the output to make sure that the core orbitals are indeed correctly treated.

For compatibility with the old code, the file should end with an empty line. The new code doesn't care about it.

For a **pseudopotential test calculation**, the format is exactly the same, except that the job code is `pt` instead of `ae`.

For a **pseudopotential generation run**, in addition to the electronic configuration chosen for the generation of the pseudopotentials (which is input in the same manner as above), one has to specify the "flavor" (generation scheme) and the set of core radii $r_c$ for the construction of the pseudowavefunction. Here is an example for Si using the Troullier-Martins scheme:

```
  pg     Si     Pseudopotential
       tm2     2.00
n=Si c=ca
      0.0       0.0       0.0       0.0       0.0       0.0
   3    3
   3    0     2.00      0.00
   3    1     0.50      0.00
   3    2     0.50      0.00
     1.12      1.35      1.17       0.0       0.0       0.0

#2345678901234567890123456789012345678901234567890123456789012345678901234567890   Ruler
```

Apart from the `pg` (pseudopotential generation) job code in the first line, there are two extra lines:

- Second line:

  Flavor and radius at which to compute logarithmic derivatives for test purposes.

  The flavor can be one of : tm2 Improved Troullier-Martins

  (format 8x, a3, f9.3)

- The last line (before the blank line) specifies, as in SIESTA and the old versions of the code:

  - The values of the $r_c$ in atomic units (bohrs) for the $s$, $p$, $d$, and $f$ orbitals. They are given by angular momentum, whatever the order of the valence orbitals in the previous lines (it is a good practice to input the valence orbitals in the order of increasing angular momentum, so that there is no possible confusion).
    (format 4f10.5)
  - Two extra fields (2f10.5) which are relevant only if partial core corrections are used (see Sect 4.3). If they are zero or blank default values are used.
  - In the most recent versions of the code an extra field (f10.5) is used for the definition of the matching radius of the partial core kinetic energy density.

  The fields are read in fixed columns: the core radii in columns 1–40, `rcore_flag` in columns 41–50, $r_{pc}$ in 51–60 and the kinetic energy density flag in 61–70. Any of the last three can be left blank. Versions of the code from 2021 to 2026 wrote these three numbers in columns 51–80 (with columns 41–50 blank); such files should be edited to move them ten columns to the left.

In the `tm2` example above, only $s$ ,$p$, and $d$ $r_c$'s are given. Here is an example for Silicon in which we are only interested in the $s$ and $p$ channels for our pseudopotential:

```
  pg     Si     s and p only
       tm2     2.00
n=Si c=ca
      0.0       0.0       0.0       0.0       0.0       0.0
   3    2
   3    0     2.00      0.00
   3    1     2.00      0.00
     1.80      1.80      0.00       0.0       0.0       0.0

#2345678901234567890123456789012345678901234567890123456789012345678901234567890   Ruler
```

The new version of the code has an extension for the case a $g$ orbital (or higher angular momentum) is present in the valence. In that case the line has one $r_c$ field for each angular momentum up to the largest one in the valence, for example five fields (format 5f10.5) for a $g$ orbital, and the core correction fields follow immediately after them (columns 51–80 in that example). Pseudopotentials are generated up to $g$ ($l = 4$).

This completes the discussion of the more common features of the input file.

# 5 Computational details

Internally the code uses Rydberg atomic units (energies in Ry, lengths in bohr). The files for CPW2000 (*Xy*`_POTKB_F.DAT`) are written in Hartree.

### 5.1 Radial grid

The radial grid points are given by

$$r(n) = \frac{e^{-A}}{Z}\Big(e^{(n-1)/B} - 1\Big), \qquad n = 1, \ldots, n_{\max}, \tag{2}$$

where $A$ and $B$ are the constants given in `atom.dat` (defaults $A = 6$, $B = 200$, printed as `a` and `b` in the output), $Z$ is the atomic number, and $n_{\max}$ is the last point with $r(n_{\max}) \le r_{\max}$, the maximum extension of the radial grid, which can also be set in `atom.dat` (default $r_{\max} = 120$ bohr). Near the origin the grid is almost linear,

$$r(n) \simeq (n-1)\frac{e^{-A}}{BZ}, \tag{3}$$

with a spacing inversely proportional to $Z$, as required by the $1s$ orbital. For large radii it is logarithmic,

$$\frac{r(n)}{r(n-1)} \simeq e^{1/B} \simeq 1 + \frac{1}{B}, \tag{4}$$

so the number of points is

$$n_{\max} \simeq 1 + B\ln\big(1 + r_{\max} Z e^{A}\big), \tag{5}$$

about 2700 for Si with the default values, and doubling $r_{\max}$ adds only $B\ln 2 \simeq 0.7B$ points. The accuracy is mostly controlled by $B$. The radial equations are integrated in the variable $n$, with $dr/dn = (r + e^{-A}/Z)/B$.

### 5.2 Solving radial equations

The non-relativistic (Schrödinger) and relativistic (Dirac, two components) radial equations are solved on the radial grid.

- At the start of the self-consistent atomic calculation no good guess of the eigenvalues is available, and the Schrödinger equation (also in relativistic calculations) is discretized by finite differences in $n$. The lowest eigenvalues and eigenvectors of the resulting tridiagonal matrix are found with the LAPACK routines `dstebz` (bisection) and `dstein` (inverse iteration). This requires no initial guess, and gives the starting eigenvalues and charge density.

- In the self-consistent iterations each bound state is found by integrating outwards from the origin up to the classical turning point, and inwards from a "practical infinity" (where $r\sqrt{V - \epsilon}$ is large) down to the same point. The two solutions are matched in value, and the eigenvalue is corrected from the discontinuity of the derivative (first-order perturbation theory), with the number of nodes used to bracket the eigenvalue.

- The integration uses a predictor-corrector method in the variable $n$: the predictor is the 5th order Adams-Bashforth and the corrector the 5th order Adams-Moulton formula. They need 5 starting points, which are obtained from the expansion of the wave-functions near the origin, $u(r) \simeq r^{l+1}(1 + a_1 r + a_2 r^2)$ for the Schrödinger equation, and $r^s(1 + a_1 r + a_2 r^2)$, $r^s(b_0 + b_1 r + b_2 r^2)$ for the major and minor components of the Dirac equation, with $s = \sqrt{\kappa^2 - (\alpha Z)^2}$. (The original code, from which this one derives, used $\alpha Z/4$ in that expansion, the factor of 2 between Hartree and Rydberg units having been applied in the wrong direction. It was found by comparing with the analytical eigenvalues of $-Z/r$.)

- The confined orbitals of the atomic basis (section 5.3) vanish at the cutoff radius $r_c$. They are found by integrating outwards up to $r_c$ and adjusting the energy (secant method and bisection, with the number of nodes) until the wave-function vanishes at $r_c$.

- The perturbative polarization orbitals are solutions of a linear inhomogeneous equation. It is solved directly with the Numerov method in the variable $n$ for $u(r)/\sqrt{dr/dn}$, integrating outwards from the origin and inwards from $r_c$, and matching at $0.9\,r_c$.

It has been checked that with increased radial grid quality the eigenvalues of $-Z/r$ converge to the analytical values, for both the Schrödinger and the Dirac equations.

### 5.3 Atomic orbitals

The code produces atomic orbitals that are used by the CPW2000 code for very good starting guesses of the plane-wave iterative diagonalization, for calculations in the atomic-orbital basis, and for the modified tight-binding (MTB) interpolation. They are written in real space to `plot_kb.dat`, with step 0.05 bohr (each one followed by a line with its basis set, zeta, kind, nodes, cutoff radius and energy), and, as Fourier transforms (section 5.4), to *Xy*`_POTKB_F.DAT`.

The atomic orbitals have angular momenta $l = 0, \ldots, l_a$, where $l_a$ is the larger of the highest occupied $l$ and $l_{\max} - 1$, $l_{\max}$ being the highest angular momentum of the pseudopotential, and the polarization orbital has $l_a + 1$ (not above $f$). For example, $l_a = 0$ for H, Li, Be and Na (polarization $p$), $l_a = 1$ for Mg, K, Ca and the $p$-block (polarization $d$), $l_a = 2$ for the transition metals and La, Ac (polarization $f$), and $l_a = 3$ for Ce–Yb, Th–U and Lu (no polarization orbital). A single-zeta (SZ) set has one orbital $\psi$ for each $l$, a double-zeta (DZ) set adds a second orbital $\phi$, and the "P" sets add one polarization orbital. The orbitals are similar to the ones used in the SIESTA code, and are of three types:

- The first orbital $\psi$ for each $l$ is a confined orbital, as the first SIESTA orbital, with a cutoff radius $r_c$ beyond which it is zero. The radial equation is first solved at an energy $\epsilon_l + \delta E$, a shift $\delta E$ above the eigenvalue $\epsilon_l$ of the valence state. The node of that solution is the "SIESTA radius" $r_S(l)$. In this code $\delta E = 0.02$ Ry (parameter `ESHIFT` in `atom_kb_basis_cutoff.f90`). The default of SIESTA was also 0.02 Ry in version 4.1, and is 0.01 Ry (0.136 eV) in version 5.4.2 (input `PAO.EnergyShift`). Where SIESTA uses $r_c = r_S(l)$, this code uses

$$r_c = \min\bigl(\max(F_l\, r_S(l), R_{\min}), R_{\max}\bigr), \tag{6}$$

  with $F_0 = 1.45$, $F_l = 1.15$ for $l > 0$, $R_{\min} = 6$ bohr and $R_{\max} = 9$ bohr. These constants minimize the difference between the atomic-orbital and plane-wave total energies of the elemental solids calculated with CPW2000, and are parameters in `atom_kb_basis_rule.f90`. The orbital $\psi$ is the solution of the radial equation, with $\psi(r_c) = 0$, and with the soft confining potential of J. Junquera, O. Paz, D. Sánchez-Portal and E. Artacho, Phys. Rev. B 64, 235111 (2001),

$$W(r) = \begin{cases} A_W \exp\Bigl(-\frac{(1-f)\,r_c}{r - f r_c}\Bigr)\frac{1}{r_c - r}, & \text{if } f r_c < r < r_c, \\ 0, & \text{otherwise,} \end{cases} \tag{7}$$

  with $A_W = 40$ Ry and $f = 0.9$, the same values as the SIESTA defaults (soft confinement is the default of SIESTA 5.4.2, but not of version 4.1).

- The second orbital $\phi$ for angular momentum $l$ differs from the SIESTA recipe ("split-norm"). It is similar to the energy derivative of the first orbital, $\partial\psi/\partial\epsilon$, in the spirit of the linearized methods (O. K. Andersen, Phys. Rev. B 12, 3060 (1975)), but vanishes at $r_c$, which $\partial\psi/\partial\epsilon$ cannot do for a confined orbital. It is the solution of

$$(H-\epsilon)\phi(r) = \psi(r) - \frac{w(r)\psi(r)}{\langle\psi \mid w \mid \psi\rangle}, \qquad \langle\psi \mid \phi\rangle = 0, \qquad \phi(r_c) = 0, \tag{8}$$

  where $H$ includes the confining potential, $\epsilon$ is the eigenvalue of $\psi$, and $w(r) = \exp(-r_c/r)$ for $r < r_c$. It is calculated as $\phi = (\partial\psi_\lambda/\partial\lambda)/\langle\psi \mid w \mid \psi\rangle$, where $\psi_\lambda$ is the confined orbital of $H + \lambda w$, by finite differences in $\lambda$ with Richardson extrapolation. It has one node and the same cutoff radius as $\psi$.

- The polarization orbital is the perturbative polarization of the first orbital $\psi$ of $l_a$, as in SIESTA (E. Artacho, D. Sánchez-Portal, P. Ordejón, A. García and J. M. Soler, Phys. Stat. Sol. (b) 215, 809 (1999)). It is the solution of

$$(H_{l_a+1} - \epsilon)\, u(r) = -r^p\, u_\psi(r), \qquad u(r_c) = 0, \tag{9}$$

  with the cutoff radius, confinement and eigenvalue $\epsilon$ of $\psi$ ($u = rR$, and $H_{l_a+1}$ is the radial hamiltonian for angular momentum $l_a + 1$). The SIESTA papers write the source with $p = 1$ (the response to an electric field), but the SIESTA code (version 5.4.2, routine `polarization` in `atom.F`) integrates the source in the grid variable without the factor $(dr/dn)^{3/2}$, which on its logarithmic grid is effectively $p = -1/2$. That gives a more compact orbital and lower total energies: for Ge (DZP, 40 Ha cutoff) the atomic-orbital energy is 102.7, 79.0 and 74.1 meV/atom above the plane-wave energy for $p = 1$, 0 and $-1/2$. The default is $p = -1/2$ (parameter `POLPOW` in `atom_kb_polarization.f90`), which reproduces the SIESTA polarization orbitals. For the elemental solids a single perturbative polarization orbital costs on average 0.7 to 2.6 mHa/atom, depending on the group of elements, compared with a confined $l_a + 1$ orbital plus its $\phi$, with three ($p$), five ($d$) or seven ($f$) fewer orbitals per atom.

A bespoke basis set (number of orbitals and cutoff radius for each $l$) can be defined in the interactive mode of `kb_conv`. A cutoff radius of 0 asks for the orbital $\phi$ of the previous orbital, and $-1$ asks for the perturbative polarization orbital.

### 5.4 Fourier transforms

The *Xy*`_POTKB_F.DAT` file contains the local potential, the Kleinman-Bylander projectors, the core and valence charge densities, the core kinetic energy density and the atomic orbitals as functions of $q = k\,\delta q$, $k = 0, \ldots, 4000$, with $\delta q = 0.01$ bohr$^{-1}$ ($q_{\max} = 40$ bohr$^{-1}$, enough for wave-function cutoffs up to 200 Ha). For an atomic orbital $R(r)$ of angular momentum $l$ the file contains

$$\tilde{R}(q) = \sqrt{\frac{2l+1}{4\pi}}\sqrt{\frac{2}{\pi}}\int_0^{r_c} r^2 R(r)\, j_l(qr)\, dr, \tag{10}$$

where $j_l$ is the spherical Bessel function. The integrals are calculated with the 8-point closed Newton-Cotes formula in the variable $n$, where the grid spacing is smaller than $0.1/q_{\max}$, and beyond that radius on a uniform grid with spacing $0.1/q_{\max}$, with the function interpolated by cubic splines.